Bridging Simulation and Experiment with Deep Neural Networks for Estimating Protein-Protein Binding Affinity

Rohan Devereux

The thesis is submitted to University College Dublin
in part fulfilment of the requirements for the degree of
BSc Applied and Computational Mathematics

School of Mathematics and Statistics
and
School of Physics
University College Dublin

Supervisor: Dr. Donal MacKernan (UCD)
Dr. Jony Castagna (STFC CECAM-UK-DARESBURY)

April 2026

Abstract

Protein–protein binding affinity measures how strongly two proteins associate and is important for protein design, therapeutic targeting, and drug discovery. Experimental measurements and high-end free-energy calculations are informative but expensive, so fast structure-based predictors remain attractive at scale. Most of those predictors estimate affinity from a single bound structure, even though binding is an ensemble property and proteins can adopt more than one relevant arrangement. This study asks whether short explicit-solvent molecular dynamics (MD) can add useful information to a residue-graph affinity predictor.

Starting from a curated wild-type subset of PPB-Affinity, each retained complex was prepared for MD, simulated in OpenMM, converted to a residue graph, and enriched with trajectory-derived summary features. The comparison used 1437 complexes and 52 models. It covered full and interface-restricted graphs, static and static-plus-dynamic features, and two model families. One path trained a masked GraphVAE and predicted affinity from graph-level latent means by ridge regression. The other used a direct supervised graph regressor. All models were evaluated over 10 outer held-out fits.

The direct supervised baseline on the full static graph performed best, with mean test RMSE 2.3379 ± 0.1067, Pearson correlation 0.5064 ± 0.0567, and $R^2 = 0.2146 \pm 0.0891$. The full graph outperformed the interface-restricted graph in all 24 matched GraphVAE comparisons, and static features outperformed static-plus-dynamic features in 11 of 12 matched GraphVAE comparisons under the static-only reconstruction target.

On this benchmark, the static whole-complex representation was strongest. The results suggest that 1.2 ns trajectories captured little new affinity-relevant motion. Coarse trajectory summaries then made any weak added signal difficult to use reliably.

Acknowledgments

I'd like to thank Dr. Donal MacKernan and Dr. Jony Castagna for supervising this thesis and providing guidance and advice. I'd also like to thank Shrinath Kumar for helping with molecular dynamics simulations using OpenMM. Finally, I'd like to thank UCD for allowing access to the Sonic HPC, without which the computationally intensive simulations used in this project would not have been possible.

Contents

1 Introduction **1**

2 Literature Review **3**

2.1 Protein–protein benchmark and evaluation 3
2.2 Representation scope 4
2.3 Dynamics and the protein–ligand comparison 4
2.4 Variational models 5
2.5 Position of this study 6

3 Methods **7**

3.1 Dataset and target 7
3.2 Molecular-dynamics preparation and simulation 8
3.3 Residue-graph construction and features 8
3.4 Graph views, analysis subset, and feature scaling 10
3.5 Models 11
3.5.1 Direct supervised baseline 12
3.5.2 Masked GraphVAE 12
3.5.3 Latent-space ridge regression 13
3.6 Repeated outer evaluation 14

4 Results **16**

4.1 Overall performance context 16
4.2 Representation scope 16
4.3 Supervision 17
4.4 Dynamic features 18
4.5 Model family 19
4.6 Stability under repeated evaluation 19
4.7 Interaction-class behaviour 20
4.8 Summary 20

5 Discussion and Conclusions **22**
5.1 Evaluation context 22
5.2 Representation scope 22
5.3 Dynamics and timescale 23
5.4 Supervision 23
5.5 Model family 24
5.6 Interaction classes 24
5.7 Limitations 24
5.8 Future work 25
5.9 Conclusion 25

A Archived Approaches **27**
A.1 Correction learning around fast affinity baselines 27
A.1.1 PRODIGY 27
A.1.2 MM/GBSA 28
A.2 The Cα contact-map approach 29
A.3 PPB-Affinity interface-attention augmentation 30

B Complete Model Matrix **32**

C Repository Layout **35**

List of Figures

3.1 Model structure used in the comparison. The two model families use the same encoder architecture. Both start with linear node and edge projections, three GINE layers, and mean-plus-max graph pooling. The supervised baseline predicts ΔG directly from the pooled graph embedding. The GraphVAE maps the pooled graph embedding to μ and $\log \sigma^2$, uses sampled latent vectors for masked reconstruction, uses the latent mean for the optional affinity head during training, and reports affinity from ridge regression on latent means computed after training from unmasked graphs with zero mask indicators. 11

4.1 Main empirical picture of the 52-model comparison. The left panel shows the top of the model ranking by mean test RMSE, with one-standard-deviation error bars over the 10 outer fits. The right panel shows matched GraphVAE factor comparisons, reported as the mean test-RMSE advantage for the first-named option. 18

4.2 Subgroup performance of the three headline models. Points show the mean outer-test metric and error bars show one standard deviation across the 10 outer fits. The model ordering is stable across subgroups, but uncertainty is visibly larger for the 105-complex TCR-pMHC subset. 20

List of Tables

2.1 Short score context for representative protein–protein affinity predictors. . 5

3.1 Composition of the full curated wild-type table and the retained 1437-complex matched subset. The retained set is enriched for antibody–antigen complexes and has a slightly more negative mean binding free energy. Percentages and ΔG summary statistics are rounded. 10

3.2 Design axes in the 52-model comparison. The direct supervised baselines cover the Cartesian product of graph view and feature mode. The remaining combinations belong to the GraphVAE family. 15

4.1 Best-performing models in the 52-model comparison. Reported RMSE, Pearson r, and R^2 values are mean $\pm$ standard deviation over the 10 outer fits. GraphVAE metrics are those obtained after ridge regression on latent means. 17

4.2 Matched GraphVAE comparisons. A positive RMSE gain means the first-named option has lower mean test RMSE. 17

4.3 Family-level mean train, validation, and test behaviour. Here $g_r = r_{\text{train}} - r_{\text{test}}$. Each row is averaged over all models in that family. 19

4.4 Pooled outer-test subgroup metrics for the best overall model (full static baseline), the best GraphVAE (full static semi-supervised GraphVAE, latent dimension 32), and the best interface model (interface static baseline). 20

A.1 Subset-specific PRODIGY results from archived curated datasets. The TCR-pMHC subset was especially weak. The small antibody–antigen subset was somewhat better. 28

A.2 Representative PRODIGY-based regression runs on the archived TCR subset. The direct model is the learned latent-space regressor. The corrected model tries to learn an adjustment to PRODIGY. 28

A.3 MM/GBSA summary from the archived approach. The estimates were far outside the scale of experimental protein–protein binding free energies, so this route was not pursued as a quantitative baseline. 29

A.4 Representative Cα contact-map regression runs on the archived TCR subset. Some variants reduced RMSE relative to raw PRODIGY, but the results were unstable and often had weak or negative test correlation. . . . 30

B.1 Complete 52-model evaluation matrix, sorted by mean test RMSE. Metrics are means over 10 outer held-out fits. 33

Chapter 1

Introduction

Protein–protein binding affinity prediction asks how strongly two proteins bind. In benchmark datasets for this task, the target is a thermodynamic quantity such as dissociation constant K_D or binding free energy ΔG, and the input is usually a single three-dimensional complex structure [1]. Accurate affinity prediction is important for protein design, therapeutic targeting, and drug discovery [2, 3]. Experimental measurements provide the ground truth, but they are too slow and expensive for large exploratory searches. Physics-based free-energy calculations aim at the same quantity, but the more faithful versions are also expensive [4, 5]. Many practical machine-learning predictors therefore estimate affinity quickly from structural input alone.

One difficulty is that binding affinity is an ensemble property. It depends on the range of shapes and relative arrangements the proteins sample, not just on one frozen structure. Proteins are dynamic, and many protein pairs also reorganise at the interface as they bind [6, 7], so a single bound complex can miss relevant information. Fast structure-based predictors therefore have to decide how much of the complex to represent. Some recent protein–protein models focus tightly on the interface, whereas others use whole-complex or hybrid representations [1, 8–10]. They also have to decide whether one bound structure is enough. Protein–ligand work has explored the use of MD beyond a single bound structure more extensively [11, 12]. Residue graphs make it possible to examine both questions in the same protein–protein setting. In this representation, amino-acid residues become nodes and residue–residue proximity or contact defines edges.

A curated wild-type, that is, non-mutant, subset of PPB-Affinity [1] provides the data. The starting curated table contained 2818 complexes, from which a matched 1437-complex subset was retained. Each retained complex went through the same preparation and short-MD procedure. DeepRank2 converted the prepared protein structure into a residue graph, and trajectory-derived summaries supplied the dynamic features. Affinity was then predicted with two model families. One path trained a masked GraphVAE, computed graph-level latent means, and predicted affinity from those latents with ridge regression. The other trained a direct supervised graph regressor on the same residue

graph. The shared setup lets graph scope, feature mode, and model family be compared on equal terms.

Four questions guide the analysis: whether a full residue graph is better than an interface-restricted representation; whether MD-derived summaries improve prediction over static graph features alone; whether a masked GraphVAE produces a better predictive representation than a direct supervised graph regressor with the same encoder architecture; and how stable those answers remain under repeated held-out evaluation.

Chapter 2 frames those questions through benchmark design, representation scope, and dynamics. Chapter 3 describes the methods. Chapter 4 answers the four questions with the 52-model comparison. Chapter 5 interprets those answers and sets out the main limitations and next steps. The appendices cover archived approaches, the complete model table, and the repository layout.

Chapter 2

Literature Review

2.1 Protein–protein benchmark and evaluation

Protein–ligand affinity prediction has had a shared structural resource since PDBbind [13]. Curated protein–protein datasets for absolute affinity remained much smaller for many years [7]. Protein–protein affinity prediction therefore developed with less data and with more variation between datasets. PPB-Affinity was introduced to improve that situation by building a larger benchmark for direct protein–protein affinity prediction [1]. Reviews of the area point to the same practical difficulties: affinity data are limited, interaction classes are mixed, and protein pairs often change shape as they bind [2, 5]. Those problems affect evaluation as well as modelling.

Single-structure scoring is restrictive in this setting. In the Kastritis benchmark, only a minority of complexes were close to rigid-body association, and large or moderate interface rearrangements were common [7]. Many protein pairs bind through bending, shifting, or local reorganisation instead of simple rigid docking. A useful predictor therefore has to cope with both shape changes within one complex and structural differences across complexes.

Benchmark design also shapes the apparent difficulty of the task because the benchmark is heterogeneous. PPB-Affinity reported Pearson $r = 0.701$ for its benchmark model on the full test set, but subgroup performance was weaker for both antibody–antigen and T-cell receptor–peptide major histocompatibility complex (TCR-pMHC) complexes [1]. The same paper linked the TCR-pMHC difficulty to strong interface similarity within that class and noted that both subgroups contain a higher proportion of mutant samples than the full dataset. Yang et al. reached a similar conclusion for antibody–antigen prediction and proposed specialised regressors for that class [14]. Different interaction classes combine different affinity ranges, interface similarities, and mutation patterns, so pooled benchmark averages can hide real differences in difficulty.

Split design changes the apparent performance of a predictor. PPI-Affinity reported

that PRODIGY reached $r = 0.74$ on a 79-complex benchmark subset but only $r = 0.31$ on a separate 90-complex hold-out set [9]. Moal et al. reduced redundancy by removing homologous complexes from evaluation [8]. In protein–ligand scoring, Graber et al. later showed that train–test leakage between PDBbind and CASF had inflated benchmark performance and that stricter splits caused clear drops for previously strong models [15]. Evaluation is therefore part of the scientific problem. Reported gains can come from model quality, split difficulty, or both.

2.2 Representation scope

Umbrella sampling and alchemical free-energy methods aim more directly at binding free energy, but they are expensive and sensitive to protocol design [4, 16]. Most of that literature is protein–ligand focused. For protein–protein systems, the same calculations are usually harder because the complexes are larger and more flexible [5]. MM/PBSA and MM/GBSA are cheaper endpoint approximations [17], but they still depend strongly on setup choices and need careful interpretation. Fast structural predictors sit much further down the cost scale, but they then have to decide how much of the complex to keep.

That scope choice runs through the protein–protein literature. PRODIGY and the PPB-Affinity benchmark model are explicitly interface-focused [1, 18]. Moal et al. and PPI-Affinity instead build descriptor sets from the complex as a whole [8, 9]. PPI-Graphomer sits between those extremes: it builds an interface-aware module, but the final predictor combines that interface representation with complete sequence and structure information [10]. Interface-focused models keep the input compact and concentrate on the contact region. Whole-complex and hybrid models retain broader structural context that may still influence affinity. The published literature does not settle that choice cleanly, which makes matched scope comparisons useful.

Even within this small set, score ranges reflect benchmark design as much as model family. Published Pearson correlations often fall roughly between 0.5 and 0.7, but those values come from different subgroup mixes, structure sources, and split rules. External or stricter evaluation can lower them sharply. Score context is useful, but direct ranking across papers is limited unless the evaluation setup is closely matched.

2.3 Dynamics and the protein–ligand comparison

Protein–ligand affinity prediction differs from protein–protein affinity prediction in several relevant ways. In protein–ligand work, the ligand is usually much smaller, and the interaction is more tightly localised to one binding pocket. Larger shared resources such as PDBbind have also been available for much longer [13]. Protein–protein systems are larger, more heterogeneous, and often require the model to decide how much of two ex-

Method	Scope	Evaluation context	Reported Pearson r
PRODIGY [9, 18]	Interfacial contacts and non-interacting surface descriptors; interface-only	79-complex benchmark subset; external 90-complex hold-out	0.74; 0.31
Moal et al. [8]	Engineered descriptors over diverse complexes; whole-complex	Diverse 137-complex benchmark	0.73
PPI-Affinity [9]	Structure-derived descriptor vector; whole-complex	Vangone–Bonvin benchmark; external 90-complex hold-out	0.62; 0.50
PPB-Affinity benchmark model [1]	128-residue interface representation; interface-restricted	Whole-dataset benchmark	0.701
PPI-Graphomer [10]	Complete sequence and structure features plus an interface-aware module; hybrid	Two external test sets	0.641; 0.625

Table 2.1: Short score context for representative protein–protein affinity predictors.

tended molecular surfaces should be retained [2, 5]. In protein–ligand prediction, pocket-restricted voxel or graph representations are often a natural choice. In protein–protein prediction, the same design can discard context that still contributes to binding strength.

Protein–ligand work gives useful perspective here. Pafnucy showed that a fixed local 3D representation could support affinity prediction when the binding problem is tightly localised [19]. More recent work has also used MD. Dynaformer argues that static structures can miss affinity-relevant information present in conformational ensembles. Timenucy and Videonucy instead learn from frame sequences instead of simple trajectory averages [11, 12]. Taken together, these studies suggest that local representations can work well when the interaction is strongly localised. They also suggest that dynamic information is more likely to help when the model keeps richer trajectory information than a small set of summary statistics.

2.4 Variational models

Variational autoencoders were introduced as latent-variable models that combine reconstruction with a regularised latent distribution [20]. Variational graph autoencoders brought the same idea to graph data [21], and later GraphVAE-style models made graph generation more explicit [22]. In molecular machine learning, related latent-space VAEs were already being used successfully for small molecules [23]. The use here is narrower than full graph generation. The GraphVAE is used to learn a compact residue-graph representation and then test whether that representation carries useful affinity signal.

Simulation work also supports this choice. Tian et al. showed that a VAE trained on protein MD trajectories could learn a latent space that captured physically plausible

structural variation [24]. That result suggests that a latent model may be useful when structure and MD features are high-dimensional and only part of that information is relevant to binding strength.

2.5 Position of this study

Three linked issues emerge from this literature. Evaluation design changes the apparent difficulty of the task. Representation scope changes how much structural context a model can use. Dynamics remains open. Protein–ligand work has moved further on MD-informed learning, but protein–protein systems are larger, more heterogeneous, and harder to benchmark at comparable scale [5, 11, 12]. The comparison in this thesis therefore varies full and interface-restricted residue graphs, static and short-MD summary features, and model family under one shared evaluation design.

Chapter 3

Methods

3.1 Dataset and target

All experiments used a curated wild-type subset of PPB-Affinity [1]. Wild-type here means non-mutant complexes. For consistency, each complex was stored with one ordered assignment of the two binding partners, labelled ligand and receptor. Curation retained one ordered ligand–receptor chain assignment per complex, removed duplicate entries, and excluded mutant complexes. A complex was kept only if a Protein Data Bank structure, an interpretable ligand/receptor chain definition, and an affinity measurement sufficient to derive a binding free energy were available. The resulting curated table contained 2818 complexes.

The interaction classes in this curated table were 199 T-cell receptor–peptide major histocompatibility complex (TCR-pMHC) complexes, 437 antibody–antigen complexes, and 2182 other or unannotated complexes. PPB-Affinity reports different affinity distributions and weaker benchmark performance for the TCR-pMHC and antibody–antigen subsets than for the full test set [1]. The dataset is therefore heterogeneous, and subgroup composition has to be tracked throughout the study.

The regression target was the standard Gibbs free energy of binding, ΔG, in kcal mol^{-1}. In this curated table, the affinity column was the reported dissociation constant K_D. All targets were therefore computed as

$$\Delta G = RT \ln K_D, \tag{3.1}$$

with $R = 0.00198720425864083$ kcal mol^{-1} K^{-1} and the temperature column `Temp_K`. Where a source temperature was unavailable, the study used 298.15 K.

3.2 Molecular-dynamics preparation and simulation

Complexes were prepared from PDB coordinates with PDBFixer and the OpenMM application layer [25, 26]. Each complex then passed through the same short-MD preparation and simulation protocol. The annotated ligand and receptor chains were extracted first, and the simulation model retained protein residues only. Missing residues were detected but not rebuilt. Nonstandard residues were converted when a standard mapping was available. Missing heavy atoms (that is, non-hydrogen atoms) were added, disulfides and cysteine states were assigned automatically, and hydrogens were added before solvation.

Prepared complexes were solvated with 1.0 nm padding and ionic strength 0.15 M. Protein parameters came from `amber14-all.xml` and water from `amber14/tip3pfb.xml`, corresponding to Amber protein parameters together with the TIP3P-FB water model [27, 28]. No per-complex pH values were available, so protonation, that is, assignment of hydrogen and charge states, was carried out at pH 7.0 for every complex.

All simulations were run in OpenMM [26] with particle mesh Ewald (PME) electrostatics, Ewald error tolerance 10^{-4}, a 1.0 nm real-space cutoff, constraints on bonds involving hydrogen, rigid water, a Monte Carlo barostat at 1 atm, a Langevin middle integrator with friction 1 ps^{-1}, and a 2 fs timestep. Each system was simulated at the temperature recorded for that complex in `Temp_K`; where no source temperature was available, the same 298.15 K fallback used for the ΔG conversion was used in simulation. Each system was minimised for up to 1000 iterations, equilibrated for 100,000 steps, and then simulated for 500,000 further steps. Coordinate frames were written every 5000 steps. Because recording began before equilibration, each complex contributed 120 sampled frames over 1.2 ns in total.

3.3 Residue-graph construction and features

Residue graphs were built with DeepRank2 [29] from the prepared protein topology written after structure preparation. Both the S and SD modes used the same post-prep graph, so the only extra information in SD came from the added trajectory summaries. DeepRank2 was run in full-complex query mode with influence radius 10^6 Å, so the whole prepared complex was retained. DeepRank2 supplied the base residue and edge descriptors for surface exposure, buried area, pair distance, and pairwise interaction terms.

The graph inputs were organised in a 2×2 layout: static versus dynamic features, each at node and edge level. Static features were present in both model modes. Dynamic features were added only in the SD mode. Because the underlying post-prep graph is shared, the incremental difference in SD is the added trajectory-derived information. That dynamic information entered only as residue- and edge-level summary features on a fixed graph. The graph itself had no temporal dimension.

- **Static node features**
  - **DeepRank2 geometry and exposure descriptors:** buried surface area, solvent-accessible surface area, residue depth, and three half-sphere exposure measures. The half-sphere exposure channels count neighbouring residues in different local directions around each residue.
  - **Neighbour counts:** numbers of nearby residues within 8 Å on the same chain and on the partner chain, counted in the first sampled MD frame by heavy-atom neighbourhood queries.
  - **Residue identity and physicochemical descriptors:** a 20-way amino-acid one-hot vector, broad residue-class indicators (non-polar, polar, charged), physicochemical scalars (side-chain size, net charge, hydrophobicity, α-helix propensity, β-sheet propensity), and N- and C-terminus flags.
- **Static edge features**
  - **Residue–residue distance:** the static distance attached to each connected residue pair in the prepared structure.
  - **Same-chain indicator:** a binary flag marking whether the two residues belong to the same protein chain.
  - **DeepRank2 interaction terms:** electrostatic and van der Waals edge descriptors supplied by DeepRank2 for each residue pair.
- **Dynamic node features**
  - **Hydration summaries:** computed from the full solvated trajectory with MDTraj [30] by counting unique water oxygen residues within 5 Å of any heavy atom of a residue in each sampled frame. The resulting features were $\log(1+\bar{w}_i)$, where $\bar{w}_i$ is the mean framewise water count for residue i, together with the standard deviation of the framewise counts.
  - **Cα flexibility:** alpha-carbon root-mean-square fluctuation (Cα RMSF), computed after least-squares alignment of the trajectory frames.
  - **Backbone torsion summaries:** frame-averaged values of $\sin\phi$, $\cos\phi$, $\sin\psi$, and $\cos\psi$, where ϕ and ψ are the standard peptide-backbone dihedral angles about the N–Cα and Cα–C bonds.
  - **Inter-chain force summaries:** coarse residue-level interaction proxies from a protein-only OpenMM analysis system rebuilt from the prepared protein topology. The original nonbonded parameters were copied into separate Coulomb and Lennard–Jones `CustomNonbondedForce` objects [26], restricted to ligand-atom versus receptor-atom interaction groups, accumulated from atom-level

Dataset	n	Other	Antibody–antigen	TCR-pMHC	Mean ΔG
Curated wild-type table	2818	2182 (77.4%)	437 (15.5%)	199 (7.1%)	-9.88 ± 2.57
Matched final subset	1437	966 (67.2%)	366 (25.5%)	105 (7.3%)	-10.12 ± 2.65

Table 3.1: Composition of the full curated wild-type table and the retained 1437-complex matched subset. The retained set is enriched for antibody–antigen complexes and has a slightly more negative mean binding free energy. Percentages and ΔG summary statistics are rounded.

forces to residue-level Euclidean norms, and stored as framewise means and standard deviations for the electrostatic and van der Waals channels.

- **Dynamic edge features**
  - **Closest heavy-atom distance summaries:** for each residue pair already connected in the static graph, MDTraj closest-heavy distances were evaluated in every sampled frame and summarised by their mean and standard deviation.
  - **Dynamic contact frequency:** for the same connected residue pairs, the fraction of sampled frames with closest-heavy distance at most 8 Å.

3.4 Graph views, analysis subset, and feature scaling

Two graph views were evaluated. The *full* view retained the complete residue graph. The *interface-restricted* view identified the closest inter-partner Cα residue pair in the first protein-only MD frame and then retained up to 128 residues nearest to that pair. Proximity for this interface-only representation was measured from one anchor coordinate per residue, using Cβ where available and Cα otherwise. The 128-residue cap follows the fixed interface-window design used in the PPB-Affinity benchmark model [1].

The matched comparison retained 1437 of the 2818 curated complexes. MD simulation, feature extraction, and repeated model evaluation were carried out on this smaller subset. The paired full/interface analysis used complexes with both graph views and the full MD-derived feature set available. The retained set contained 105 TCR-pMHC, 366 antibody–antigen, and 966 other complexes. Table 3.1 compares the retained set with the full curated table.

Feature scaling was fit separately inside each outer split and only on that split's training complexes. For each node or edge feature, all training values were pooled across residues or edges, truncated to the training 0.5th and 99.5th percentiles, centred by the training mean, scaled by the training standard deviation, and finally clipped to the z-score range $[-8, 8]$. The same frozen transformation was then applied to the validation and test complexes.

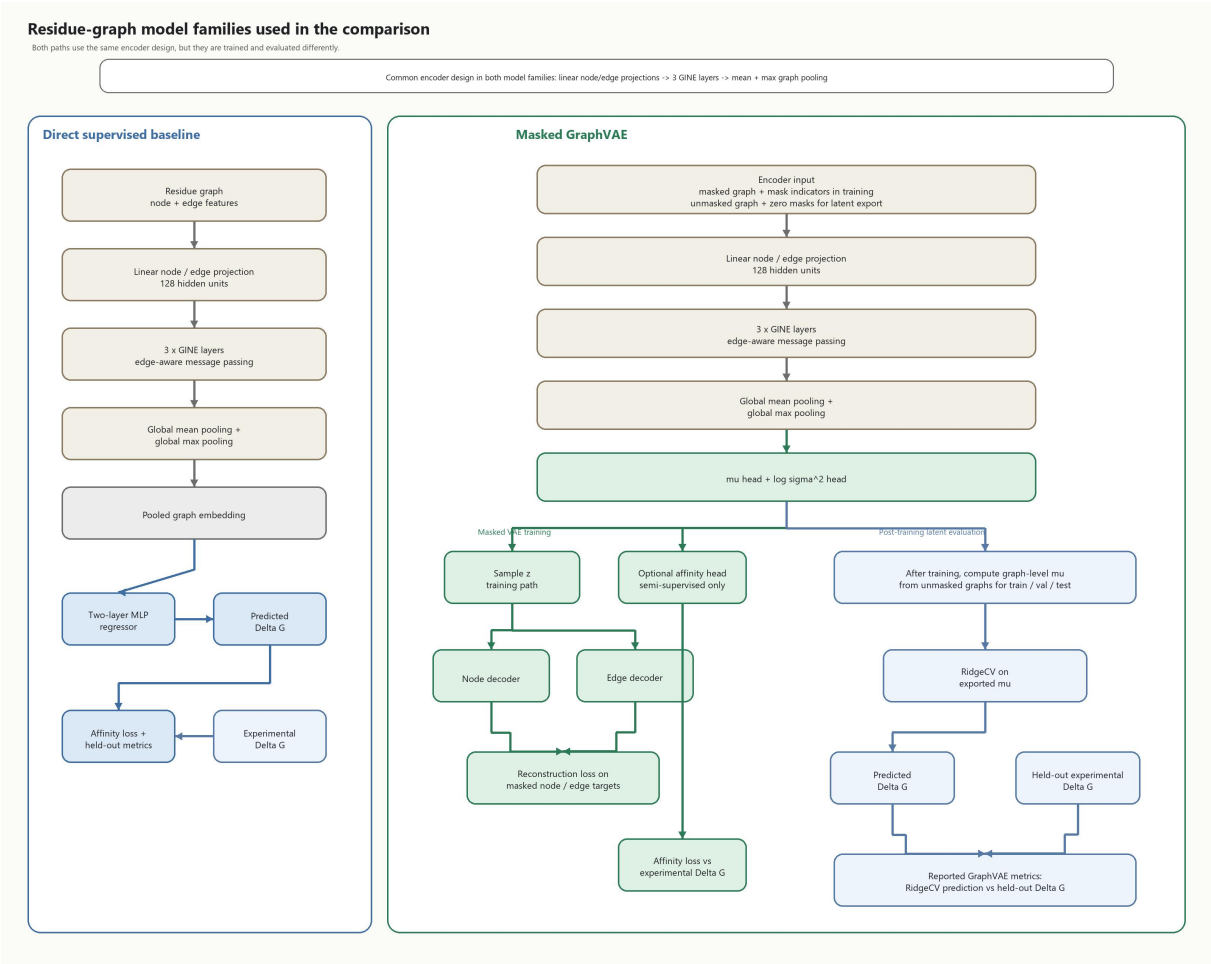

Figure 3.1: Model structure used in the comparison. The two model families use the same encoder architecture. Both start with linear node and edge projections, three GINE layers, and mean-plus-max graph pooling. The supervised baseline predicts ΔG directly from the pooled graph embedding. The GraphVAE maps the pooled graph embedding to μ and $\log \sigma^2$, uses sampled latent vectors for masked reconstruction, uses the latent mean for the optional affinity head during training, and reports affinity from ridge regression on latent means computed after training from unmasked graphs with zero mask indicators.

3.5 Models

Both model families used the same residue-graph encoder architecture. The supervised baseline predicted affinity directly from the pooled graph representation. The GraphVAE learned a masked latent representation, and affinity was then predicted from the latent means with ridge regression.

Figure 3.1 shows the shared encoder backbone and the two model paths used in the comparison.

3.5.1 Direct supervised baseline

The supervised baseline was a residue-graph regressor in PyTorch Geometric [31]. Node and edge inputs were projected linearly to 128 hidden units, passed through three graph isomorphism network layers with edge features (GINE), pooled with the concatenation of global mean pooling and global max pooling, and mapped to ΔG by a two-layer multilayer perceptron [32, 33]. In each GINE layer, a residue updates its hidden representation by aggregating information from neighbouring residues together with the features on the edges that connect them. Stacking three such layers lets the encoder combine local contact information with a wider structural context before graph pooling. Training used mean-squared error loss, AdamW with learning rate 10^{-4} and weight decay 10^{-5}, batch size 16, maximum 120 epochs, and early stopping after 20 epochs without validation improvement. The selected checkpoint was the one with the lowest validation MSE. These settings were kept fixed across all four baseline variants.

3.5.2 Masked GraphVAE

The GraphVAE used the same encoder architecture as the supervised baseline: linear node and edge projections to 128 hidden units, three GINE layers, and mean-plus-max graph pooling. Two linear heads mapped the pooled representation to the latent mean μ and log-variance $\log \sigma^2$. Predicted log-variances were clipped to the interval $[-8, 8]$ before sampling and Kullback–Leibler evaluation. The latent dimensionality was one of $d_z \in \{8, 16, 32\}$. The masking task trains the model to fill in deliberately hidden feature values from the rest of the graph. The goal is to learn a compact graph representation for affinity prediction.

For every batch, target node entries and target edge entries were masked independently with probability 0.30. The encoder received the masked graph together with binary mask indicators. The node decoder received the graph latent, the unmasked non-target node inputs, the masked target node inputs, and the node-mask indicators. The edge decoder received the analogous edge quantities. The reconstruction task was defined feature by feature for selected node and edge channels, instead of asking the model to generate an entire graph from scratch [21, 22].

The reconstruction targets used in the 52-model comparison were:

- **Static-only target:** buried surface area, solvent-accessible surface area, residue depth, the three half-sphere exposure channels, same-chain and partner-chain neighbour counts within 8 Å, and edge distance.

- **Dynamic-only target:** hydration, RMSF, torsion, and force node features together with all dynamic edge features.

- **Static-plus-dynamic target:** the union of the static target and dynamic target sets.

Residue-identity indicators and the same-chain edge flag were input features only; they were excluded from the reconstruction targets.

For a batch of graphs, the GraphVAE objective was

$$\mathcal{L} = \mathcal{L}_{\text{node}} + \mathcal{L}_{\text{edge}} + \beta \mathcal{L}_{\text{KL}} + 0.01\, \mathcal{L}_{\text{corr}} + \lambda_{\text{aff}} \mathcal{L}_{\text{aff}}, \tag{3.2}$$

where β is the weight on the Kullback–Leibler regulariser, $\mathcal{L}_{\text{node}}$ and $\mathcal{L}_{\text{edge}}$ were masked mean-squared reconstruction losses, $\mathcal{L}_{\text{corr}}$ penalised the squared off-diagonal entries of the latent covariance matrix to discourage redundant latent coordinates, and $\mathcal{L}_{\text{aff}}$ was an MSE loss on a latent-MLP affinity head. For this head, ΔG was normalised by the mean and standard deviation of the training targets in that outer split. Unsupervised runs used $\lambda_{\text{aff}} = 0$. Semi-supervised runs used $\lambda_{\text{aff}} = 1$. The Kullback–Leibler term was

$$\mathcal{L}_{\text{KL}} = \frac{1}{B} \sum_{b=1}^{B} \frac{1}{d_z} \sum_{j=1}^{d_z} \left[-\frac{1}{2} \left(1 + \log \sigma_{bj}^2 - \mu_{bj}^2 - \sigma_{bj}^2 \right) \right], \tag{3.3}$$

where μ_{bj} is the latent mean for graph b and latent coordinate j, and σ_{bj}^2 is the corresponding latent variance. The model predicts μ and $\log \sigma^2$, so σ^2 is obtained by exponentiating the predicted log-variance. Here B is the batch size and d_z is the latent dimensionality, so the loss averages the Kullback–Leibler penalty over graphs and latent coordinates.

All GraphVAE models were trained with AdamW, learning rate 10^{-4}, weight decay 10^{-5}, batch size 16, maximum 120 epochs, and early-stopping patience 20. Gradient norms were clipped at 5 during optimisation. The KL weight β was annealed linearly from 0 to 1 over the first 30% of training epochs, following the KL cost annealing or KL warm-up strategy used to stabilise variational training [34, 35]. Checkpoint selection used validation reconstruction loss for unsupervised runs and validation reconstruction loss plus $\lambda_{\text{aff}} \mathcal{L}_{\text{aff}}$ for semi-supervised runs.

3.5.3 Latent-space ridge regression

GraphVAE results are reported from a linear ridge regressor fit to the latent means. The neural affinity head was used during semi-supervised training, but the reported GraphVAE numbers come from one shared linear downstream model so that the comparison stays focused on the information already present in the learned representation. After training, one graph-level latent-mean vector was obtained per complex for the train, validation, and test splits. These latent means were computed from the unmasked standardized graphs, with the mask-indicator channels set to zero.

For training complexes with latent matrix $\mathbf{Z}_{\text{train}}$ and target vector $\mathbf{y}_{\text{train}}$, the ridge model solves

$$(\hat{\mathbf{w}}, \hat{b}) = \arg\min_{\mathbf{w},b} \|\mathbf{y}_{\text{train}} - \mathbf{Z}_{\text{train}}\mathbf{w} - b\mathbf{1}\|_2^2 + \alpha\|\mathbf{w}\|_2^2, \tag{3.4}$$

where α controls the strength of ℓ_2 regularisation and b is the fitted intercept. No further scaling was applied to the latent means before this regression step. The regularisation strength was selected by 5-fold inner cross-validation on the training latents with `RidgeCV` from scikit-learn [36], searching the grid $\{10^{-4}, 3\times10^{-4}, 10^{-3}, 3\times10^{-3}, 10^{-2}, 3\times10^{-2}, 10^{-1}, 3\times10^{-1}, 1, 3, 10, 30, 100\}$. The resulting fitted linear model was then applied without refitting to the train, validation, and test complexes in that outer split. The headline GraphVAE metrics in Chapter 4 are the held-out metrics of this ridge regression.

3.6 Repeated outer evaluation

The 52-model comparison was evaluated on the matched 1437-complex set with 5 outer folds and 2 repeats. Repeated outer resampling was used because a single split can give a misleading picture of comparative performance when many closely related models are tested [37]. Resampling operated at the level of complexes, so all trajectory frames from a complex stayed in the same split. In each outer fit, one fold was held out as test and the remaining four folds were split into training and validation sets with validation fraction 0.1875 of the training-plus-validation pool, giving an overall train/validation/test proportion of approximately 65/15/20. In practice, the outer fits contained 934 training complexes, 215 or 216 validation complexes, and 287 or 288 test complexes. At each outer fit, feature scaling, model training, checkpoint selection, and latent ridge fitting were all carried out within that split. Split generation and model fitting used deterministic pseudorandom seeds derived from the outer repeat and fold from a base seed of 2026.

The comparison contained 52 models. Table 3.2 summarises the design axes. Forty-eight models were GraphVAEs and four were direct supervised baselines. Throughout the results chapter, S denotes static-only inputs, SD denotes static-plus-dynamic inputs, and d_z or z denotes latent dimension.

Held-out performance was measured with RMSE, Pearson correlation, and R^2. Every metric reported in Chapter 4 is the arithmetic mean over the 10 outer fits for the corresponding model, and the corresponding standard deviations are reported from the same ten fits.

Axis	Options	Applies to
Graph view	Full; interface-restricted	All 52 models
Feature mode	S (static-only); SD (static-plus-dynamic)	All 52 models
Model family	Direct supervised baseline; GraphVAE with latent-space ridge regression	All 52 models
Supervision	Unsupervised; semi-supervised	GraphVAE models only
Reconstruction target	Static-only; static-plus-dynamic; dynamic-only	SD GraphVAE models only
Latent dimension	8; 16; 32	GraphVAE models only

Table 3.2: Design axes in the 52-model comparison. The direct supervised baselines cover the Cartesian product of graph view and feature mode. The remaining combinations belong to the GraphVAE family.

Chapter 4

Results

The analysis used the 1437-complex subset for which paired full and interface-restricted graph views were available. It covered 52 models: 48 GraphVAE variants and 4 direct supervised baselines. Every reported metric is the mean over 10 outer held-out fits from 5 folds and 2 repeats. Here S denotes static-only inputs, SD static-plus-dynamic inputs, *semi* semi-supervised GraphVAE training, *unsup* unsupervised GraphVAE training, and z latent dimension. The complete 52-model table is given in Appendix B.

4.1 Overall performance context

The full static supervised baseline ranked first overall. Among the GraphVAE models, the best result came from the full static semi-supervised model with latent dimension 32. Among the interface-restricted models, the static supervised baseline performed best. Figure 4.1 and Table 4.1 list the ranking and headline values.

The best supervised baseline outperformed the best GraphVAE by 0.0913 RMSE units and 0.0415 Pearson-correlation units on the mean metrics. Across the 10 outer fits, it had lower test RMSE in 8 of 10 splits than the best GraphVAE. The strongest result came from direct supervised prediction on the full static graph.

4.2 Representation scope

Graph view had the strongest effect in the matched GraphVAE comparisons. The full graph outperformed the interface-restricted graph in all 24 matched pairs. This was the clearest and most stable directional result in the entire matrix. Table 4.2 lists the numerical summary, and Figure 4.1 shows the same comparisons visually.

The full-over-interface effect was also visible in the baseline models. The full static baseline ranked first overall. The interface static baseline was more than 0.24 RMSE units worse on the mean test metric. The same ordering held across the antibody–antigen, TCR-

Model	Test RMSE	Test r	Test R^2	Test−Train RMSE
Full baseline, S	2.3379 ± 0.1067	0.5064 ± 0.0567	0.2146 ± 0.0891	0.8021
Full baseline, SD	2.4073 ± 0.1290	0.4802 ± 0.0494	0.1685 ± 0.0876	0.8414
Full GraphVAE, S, semi, $z = 32$	2.4292 ± 0.0809	0.4649 ± 0.0363	0.1537 ± 0.0674	0.7451
Full GraphVAE, SD, semi, static-only, $z = 8$	2.4364 ± 0.1065	0.4749 ± 0.0529	0.1478 ± 0.0860	1.1303
Interface baseline, S	2.5782 ± 0.1120	0.3451 ± 0.0572	0.0470 ± 0.0820	0.8090
Interface GraphVAE, S, semi, $z = 8$	2.6041 ± 0.1019	0.3402 ± 0.0611	0.0282 ± 0.0718	0.8018

Table 4.1: Best-performing models in the 52-model comparison. Reported RMSE, Pearson r, and R^2 values are mean ± standard deviation over the 10 outer fits. GraphVAE metrics are those obtained after ridge regression on latent means.

Comparison	Pairs	Mean RMSE gain for first option	Mean Pearson r gain for first option	Pair wins for first option
Full over interface	24	0.1005	0.0993	24 / 24
S over SD (static-only target)	12	0.0254	0.0215	11 / 12
Semi over unsup, full S	3	0.1640	0.2497	3 / 3
Semi over unsup, full SD	9	0.1604	0.3339	9 / 9
Semi over unsup, interface S	3	0.0290	0.2436	3 / 3
Semi over unsup, interface SD	9	-0.0081	0.2766	3 / 9

Table 4.2: Matched GraphVAE comparisons. A positive RMSE gain means the first-named option has lower mean test RMSE.

pMHC, and broad other subgroups. For this benchmark and this graph family, keeping broader structural context was more useful than restricting the representation to a fixed interface window.

4.3 Supervision

Supervision changed the GraphVAE results more than latent dimension or reconstruction target. In the matched comparisons, semi-supervision improved every full-graph pair and every interface static pair. The interface dynamic pairs were mixed, but mean correlation still increased. The clearest difference is between the unsupervised and semi-supervised family averages in Table 4.3. For the full static family, mean test correlation rose from 0.2055 in the unsupervised runs to 0.4551 in the semi-supervised runs. In the interface static family, it rose from 0.0925 to 0.3361. The same pattern holds for RMSE.

The unsupervised models therefore learned latent spaces that were weak predictors of affinity on their own. Adding the auxiliary affinity objective made those latents much more useful for the downstream ridge regressor. In this comparison, the latent representation did not become affinity-relevant automatically through masked reconstruction. It became much more affinity-relevant once training was biased toward the binding target.

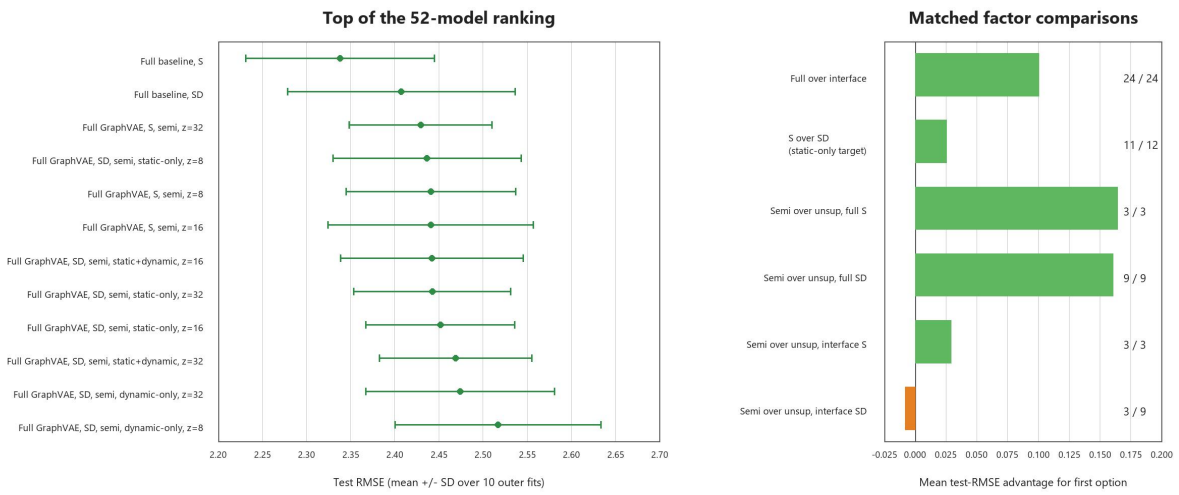

Figure 4.1: Main empirical picture of the 52-model comparison. The left panel shows the top of the model ranking by mean test RMSE, with one-standard-deviation error bars over the 10 outer fits. The right panel shows matched GraphVAE factor comparisons, reported as the mean test-RMSE advantage for the first-named option.

4.4 Dynamic features

Dynamic features did not improve held-out prediction. The supervision results show that the unsupervised families carried little affinity signal, so the effect of dynamic features is more informative in the semi-supervised GraphVAE families and in the direct baseline comparison. Under the static-only reconstruction target, the static representation outperformed the static-plus-dynamic representation in 11 of 12 matched GraphVAE comparisons. Within the full semi-supervised SD family, the reconstruction target also changed the ranking of the dynamic models. Averaged over latent dimensions 8, 16, and 32, the mean test RMSE was 2.4434 for the static-only target, 2.4795 for the static-plus-dynamic target, and 2.5150 for the dynamic-only target. The corresponding mean test Pearson correlations were 0.4682, 0.4335, and 0.4221.

Table 4.3 separates in-sample fit from held-out prediction. In the semi-supervised families, adding dynamic summaries often increased train correlation and reduced train RMSE, yet those gains faded on validation and test. The clearest example is the full semi-supervised GraphVAE family: moving from S to SD raised mean train correlation from 0.7494 to 0.8218 and lowered mean train RMSE from 1.7385 to 1.4744, yet mean test correlation fell from 0.4551 to 0.4413 and mean test RMSE worsened from 2.4369 to 2.4793. The same pattern appears in the interface semi-supervised families. The dynamic summaries gave the models more variation to fit, but that extra fit stayed on the training set instead of improving held-out prediction.

In the semi-supervised families, dynamic features changed training fit more than held-out prediction, and their main effect was to increase overfitting.

Family	Train r	Val r	Test r	g_r	Train RMSE	Test RMSE
Full baseline S	0.8225	0.5337	0.5064	0.3161	1.5358	2.3379
Full baseline SD	0.8077	0.5191	0.4802	0.3275	1.5659	2.4073
Full VAE unsup S	0.2486	0.1897	0.2055	0.0431	2.5734	2.6009
Full VAE unsup SD	0.1809	0.0967	0.1074	0.0736	2.6062	2.6398
Full VAE semi S	0.7494	0.4666	0.4551	0.2942	1.7385	2.4369
Full VAE semi SD	0.8218	0.4580	0.4413	0.3805	1.4744	2.4793
Interface VAE unsup S	0.1495	0.0794	0.0925	0.0570	2.6289	2.6402
Interface VAE unsup SD	0.1261	0.0382	0.0424	0.0838	2.6324	2.6538
Interface VAE semi S	0.7083	0.3356	0.3361	0.3722	1.8467	2.6112
Interface VAE semi SD	0.7685	0.3231	0.3189	0.4496	1.6479	2.6620

Table 4.3: Family-level mean train, validation, and test behaviour. Here $g_r = r_{\text{train}} - r_{\text{test}}$. Each row is averaged over all models in that family.

4.5 Model family

The direct supervised baseline was stronger than the best GraphVAE on held-out prediction. The best full static baseline achieved mean test RMSE 2.3379 ± 0.1067. The best GraphVAE achieved 2.4292 ± 0.0809. The baseline had lower test RMSE in 8 of the 10 outer fits. This ordering suggests that the shared encoder was strong enough to support both model families, but that direct supervised training made better use of it for the affinity task. The GraphVAE remained competitive, especially in the full static semi-supervised family, but the extra reconstruction objective did not translate into a stronger final predictor.

4.6 Stability under repeated evaluation

The main qualitative conclusions stayed stable across repeated evaluation, although the margin between the two best full-graph models was modest. The full static baseline had lower test RMSE than the interface static baseline in 9 of the 10 outer fits. The full-over-interface effect was clearer again in the matched design comparisons, where the full graph won all 24 GraphVAE pairs.

The same ranking and the same directional factor effects recurred across multiple train/test partitions. The full static baseline and the best full static GraphVAE stay close on a few splits, but the overall ordering and the sign of the main factor comparisons did not change.

Subgroup	n	Full baseline RMSE	Full baseline r	Full GraphVAE RMSE	Full GraphVAE r	Interface baseline RMSE	Interface baseline r
Antibody–antigen	366	2.048	0.292	2.134	0.193	2.356	0.089
Other	966	2.501	0.430	2.584	0.394	2.675	0.312
TCR-pMHC	105	1.663	0.466	1.879	0.406	2.440	0.333

Table 4.4: Pooled outer-test subgroup metrics for the best overall model (full static baseline), the best GraphVAE (full static semi-supervised GraphVAE, latent dimension 32), and the best interface model (interface static baseline).

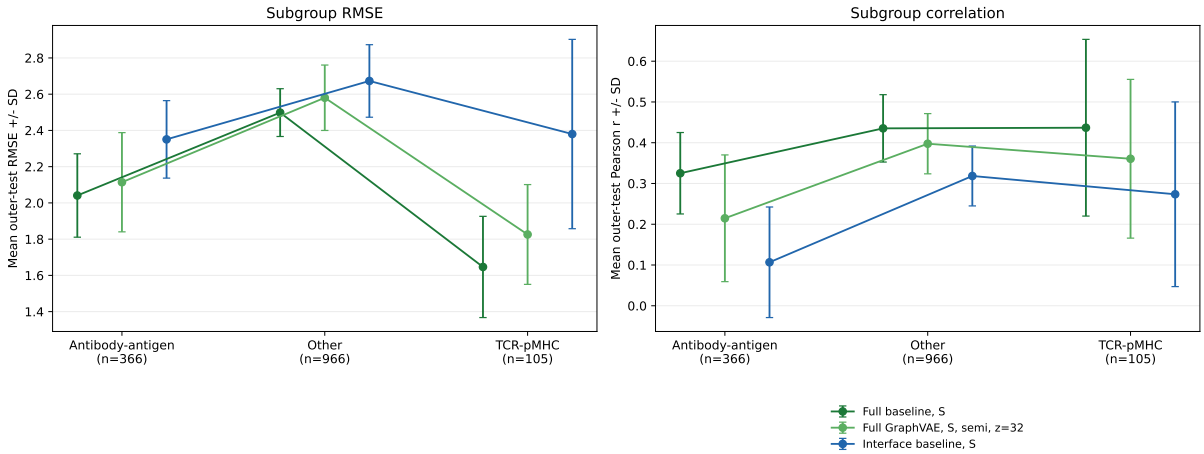

Figure 4.2: Subgroup performance of the three headline models. Points show the mean outer-test metric and error bars show one standard deviation across the 10 outer fits. The model ordering is stable across subgroups, but uncertainty is visibly larger for the 105-complex TCR-pMHC subset.

4.7 Interaction-class behaviour

The matched 1437-complex analysis set contained 966 complexes in the broad *other* class, 366 antibody–antigen complexes, and 105 TCR-pMHC complexes. For subgroup analysis, the outer-test predictions were pooled over all 10 fits, so each complex contributes two held-out predictions, one from each repeat.

The subgroup results followed the same ordering. The full static baseline remained the strongest of these models in all three interaction classes. The full static GraphVAE remained second, and the interface baseline remained weaker in every subgroup. The weakest correlations were in the antibody–antigen subgroup. The TCR-pMHC subgroup had the smallest sample size and therefore the widest uncertainty. Table 4.4 and Figure 4.2 give the subgroup values and outer-fit variability.

4.8 Summary

The 52-model comparison answers the four study questions consistently. Whole-complex graphs were stronger than interface-restricted graphs. Dynamic features offered no held-out gain over the static representation, even though they sometimes improved training fit. The direct supervised baseline on the full static graph was the strongest predictor

overall. Those same conclusions persisted across repeated outer splits and across the antibody–antigen, TCR-pMHC, and broad other interaction classes.

Chapter 5

Discussion and Conclusions

5.1 Evaluation context

Published protein–protein predictors have reported higher correlations on their own evaluation sets than the best model reached here. Moal et al. reported $r = 0.73$ on a diverse 137-complex benchmark, PPB-Affinity reported $r = 0.701$ on its full test set, and PPI-Graphomer reported $r = 0.641$ and $r = 0.625$ on two external sets [1, 8, 10]. The best model here reached $r = 0.506$ on the matched 1437-complex subset. The main contribution is methodological. It lies in a controlled comparison of representation scope, dynamic features, and model family within one benchmark, one graph-construction procedure, and one repeated held-out evaluation design.

That evaluation design is part of the contribution. Earlier work showed that score changes can come from the split as well as from the model [8, 9, 15]. The repeated 2×5 outer evaluation used here reduces the chance that one favourable split will dominate the conclusion. Across the held-out fits, the same broad ordering and the same factor effects recur even though the top two full-graph models remain fairly close on a few splits.

5.2 Representation scope

The full residue graph outperformed the interface-restricted representation. The full view won all 24 matched GraphVAE comparisons and both supervised baseline comparisons. Whole-complex context appears to help the model recognise which partners are present, how the two chains are arranged in space, and what broader structural setting surrounds the binding site. A fixed 128-residue interface window keeps only part of that information. On this benchmark, the extra context was more useful than a tighter focus on the contact region. The pattern is closer to whole-complex or hybrid approaches such as Moal et al., PPI-Affinity, and PPI-Graphomer than to strongly interface-focused predictors such as PRODIGY and the PPB-Affinity benchmark model [1, 8–10, 18]. Under the same encoder

family and repeated held-out evaluation, the whole-complex view was the stronger choice.

5.3 Dynamics and timescale

The static feature set remained stronger than the dynamic features. The comparison between S and SD was intentionally controlled. Both modes started from the same post-prep graph, so the only extra information in SD came from the trajectory summaries. Within that shared setup, the family-level results show the same pattern in both the full and interface semi-supervised families. Adding dynamic features improved train fit but weakened validation and test performance. In these families, the extra dynamic information increased overfitting without improving held-out prediction.

Trajectory length is the most likely reason. Each complex contributed 1.2 ns of saved MD, which is enough to capture local fluctuation and contact persistence but is short relative to many rearrangements that shape protein binding [5–7]. Slow collective motions often extend to microseconds and slower [6]. These trajectories were therefore unlikely to sample much of the slower motion that distinguishes one binding ensemble from another. Limited added signal is the most plausible explanation for the weak dynamic-feature results.

Feature design is another part of the story. Recent protein–ligand work suggests that MD becomes more useful when the model keeps richer temporal or ensemble information instead of reducing each trajectory to a few averages [11, 12]. Here, each short trajectory was compressed to residue- and edge-level summary features on a fixed graph. That decision made the comparison tractable at scale, but it also meant that temporal order was not represented in the graph at all and that each dynamic channel was reduced to a small set of moments or frequencies. The dynamic idea remains plausible. These summaries did not retain enough stable new signal to improve affinity prediction.

5.4 Supervision

Semi-supervision made the GraphVAE latents much more useful for affinity prediction than masked reconstruction alone. The unsupervised families had low correlations on train, validation, and test, together with small train–test gaps. That pattern is better explained by weak affinity signal in the learned latent space than by simple overfitting. Once the affinity objective was added, the same encoder family produced much stronger downstream correlations in nearly every matched comparison.

This result suggests that the masked reconstruction task was learning graph structure, but not organising the latent space in a way that was strongly aligned with binding affinity. The auxiliary affinity loss changed that alignment. In this study, the latent representation

did not become affinity-relevant automatically. It became much more affinity-relevant once training was biased toward the target.

5.5 Model family

The masked GraphVAE remained competitive, and the best semi-supervised variants were close to the strongest baseline. The full static baseline still produced the strongest held-out predictor. A likely explanation is that direct supervised training can spend all of its capacity on affinity, whereas the VAE also has to spend capacity reconstructing masked graph features. In this comparison, that trade-off left the direct supervised path with the stronger affinity predictor. Variational models can still learn useful protein latent spaces in other settings [20, 21, 24].

5.6 Interaction classes

Interaction classes also differed meaningfully. On pooled outer-test predictions, the best overall model, the full static baseline, gave Pearson $r = 0.292$ on antibody–antigen, $r = 0.430$ on the broad other class, and $r = 0.466$ on TCR-pMHC. The full static baseline remained the strongest of the main models in all three interaction classes. This matches PPB-Affinity's subgroup behaviour and later antibody-specific work arguing that antibody–antigen complexes benefit from specialised treatment [1, 14]. A pooled benchmark average can hide those differences. A model can look competitive overall and still struggle on an interaction class of practical interest. The antibody–antigen correlations here are lower than in the broad other class. The TCR-pMHC subgroup is also small enough that its uncertainty remains large.

5.7 Limitations

Time and compute shaped the scale of the study. An ideal version of this comparison would have carried the full 2818-complex curated table through longer and more carefully tailored simulations. That was not feasible here because explicit-solvent MD at this scale was too slow for the available budget. The final controlled comparison therefore used a 1437-complex matched subset and 1.2 ns trajectories per complex. PPB-Affinity itself reported stronger performance as benchmark scale increased [1], so the reduced analysed set likely limited absolute predictive power even before the MD question is considered.

The dynamic features also carry noise from how the simulations had to be prepared and summarised. To keep enough complexes in the study, preparation was standardised across complexes, missing residues were not rebuilt, protonation was standardised, and

each short trajectory was compressed to residue- and edge-level summary statistics on a fixed graph. Those choices made the comparison feasible, but they also combine thermal fluctuation, finite sampling, and summary-statistic noise. Using the same post-prep graph for both S and SD reduced noise in the comparison itself, because both modes started from the same structure source. That same choice may also leave the absolute static-only numbers below what graphs built directly from raw experimental coordinates could achieve.

The retained 1437-complex set is enriched for antibody–antigen complexes and slightly stronger binders, so the reported numbers apply to this matched subset and not to the full PPB-Affinity benchmark. The repeated resampling was done by complex identifier only, leaving homology-aware and structure-aware separation unconstrained, so related protein families could still appear across train and test. Earlier protein–protein studies removed homologous complexes for this reason, and similar leakage problems have been documented in protein–ligand affinity prediction [8, 15]. Transfer to unbound or predicted inputs remains open because only bound complex structures were used here [2].

5.8 Future work

A stronger test of dynamics would use longer or better-targeted trajectories, especially if the goal is to capture slower binding-related motion as well as local fluctuation. More careful structure preparation would also help, but that would require substantially more manual preparation work across a benchmark of this size. Recent protein–ligand work suggests that MD becomes more useful when the model keeps richer temporal or ensemble information instead of reducing the trajectory to a few averages [11, 12].

Richer structure and evaluation settings would also be useful. One shared post-prep graph was used for both S and SD so that the incremental effect of the trajectory summaries stayed isolated. Comparing that controlled design with graphs built directly from raw experimental coordinates would show how much the shared post-prep source affects absolute static performance. Stricter family-aware splits and the same representation choices on unbound or predicted structures would also be useful, since practical affinity prediction is often harder in those settings.

5.9 Conclusion

On this benchmark, a whole-complex static residue graph was the strongest starting point for protein–protein affinity prediction. In the 52-model comparison, graph scope had a larger effect on performance than the dynamic features added here. The full static baseline beat every other model. The dynamic variants more often improved training fit than held-out prediction. For this benchmark, strong whole-complex static models and

careful repeated evaluation are the best starting point. Dynamics become more attractive when longer simulations or models that retain temporal structure can justify the extra complexity.

Appendix A

Archived Approaches

This appendix summarises the main alternative approaches explored before the residue-graph study reported in the main chapters. The aim is to show what each branch was trying to test, what the provisional results suggested, and why project focus later shifted toward the final residue-graph comparison. The archived materials remain available under `ARCHIVE` in the repository.

A.1 Correction learning around fast affinity baselines

An early idea was to predict a correction

$$\delta = \Delta G_{\text{exp}} - \Delta G_{\text{base}}$$

instead of predicting ΔG directly. That strategy only makes sense if the baseline already captures a meaningful part of the affinity signal. Two approaches were explored in this form. One used the contact-based baseline PRODIGY [18]. The other used endpoint free-energy estimates from MM/GBSA [17]. In practice, neither baseline met that requirement, so later work shifted toward predicting absolute binding affinity directly.

A.1.1 PRODIGY

One approach used PRODIGY estimates as a fast structural baseline and then learned a correction to experimental affinity [18]. The idea was plausible because PRODIGY is quick, interpretable, and already tailored to protein–protein complexes, so a learned model might only need to capture the systematic error left over after a reasonable first estimate. On the archived TCR-heavy subsets, however, that starting point was weak, and direct prediction from learned structural representations was stronger than learning a correction around PRODIGY. That pattern is consistent with later evidence that PRODIGY can look much stronger on easier benchmark subsets than on separate hold-out evaluation sets

Archived subset	n	RMSE	MAE	Pearson r	Spearman ρ
TCR-pMHC	65	9.19	7.12	−0.051	−0.044
Antibody–antigen	70	2.33	1.80	0.243	0.236
Diverse100 mixed set	100	6.75	4.66	−0.175	−0.086

Table A.1: Subset-specific PRODIGY results from archived curated datasets. The TCR-pMHC subset was especially weak. The small antibody–antigen subset was somewhat better.

Archived regression variant	PRODIGY n	PRODIGY RMSE	PRODIGY r	Direct RMSE	Direct r	Corrected RMSE
Early contact-map latent regressor	13	10.50	−0.268	6.92	−0.018	8.64
Set-based edge encoder, 24 epochs	13	10.50	−0.268	2.80	−0.500	7.75

Table A.2: Representative PRODIGY-based regression runs on the archived TCR subset. The direct model is the learned latent-space regressor. The corrected model tries to learn an adjustment to PRODIGY.

[9]. The likely reason is that a contact-based baseline is most useful when it already tracks the broad structure of the target. On the harder subsets here, especially the TCR-rich ones, the baseline was already too weak for the residual to become a simpler learning problem.

The subset-specific baseline results showed the same pattern. On an archived TCR-only dataset, PRODIGY was essentially uncorrelated with experiment. On a small antibody–antigen dataset, the same baseline was better, though still modest. A mixed 100-complex subset was again weak.

Table A.2 then compares two learned routes on overlapping archived TCR test slices. One predicts affinity directly from the learned representation. The other predicts a correction around PRODIGY. The direct route reduced RMSE more consistently, which made the correction-learning idea less attractive for this benchmark.

A.1.2 MM/GBSA

MM/GBSA endpoint estimates provided a second correction-learning baseline [17]. This idea was also reasonable at first sight, because a more explicitly physics-based baseline might have captured energetic structure that a contact heuristic could not, leaving a smaller and more learnable residual. The archived MM/GBSA estimates were too poorly calibrated for that role. On 163 comparable estimates, Pearson correlation was −0.127, Spearman correlation was −0.096, RMSE was 2458.69, and MAE was 1398.31. More importantly, the predicted values lay far outside the experimental target range: the median estimate was 466.38 kcal mol^{-1}, the 95th percentile was 5590.71 kcal mol^{-1}, and the maximum exceeded 13 000 kcal mol^{-1}, whereas the corresponding experimental free energies lay roughly between −21 and −4 kcal mol^{-1}. That scale failure suggests a mismatch between the method and the present benchmark, not just ordinary noise. The interpre-

Quantity	Value
Comparable estimates	163
Pearson correlation	-0.127
Spearman correlation	-0.096
RMSE	2458.69
MAE	1398.31
Median predicted MM/GBSA ΔG	466.38
95th percentile predicted MM/GBSA ΔG	5590.71
Maximum predicted MM/GBSA ΔG	13628.15

Table A.3: MM/GBSA summary from the archived approach. The estimates were far outside the scale of experimental protein–protein binding free energies, so this route was not pursued as a quantitative baseline.

tation is more straightforward for this branch. For protein–protein binding, endpoint MM/PBSA or MM/GBSA methods are already challenging because the binding free energy is obtained as a small residual of large energetic contributions and omitted entropic terms can change the estimate substantially [5, 17]. In this branch the calculation was also kept deliberately lightweight, using a GB endpoint setup on short subsampled trajectories without an explicit entropy correction, so the branch never produced a baseline that was quantitatively useful for correction learning.

Taken together, the two baseline branches answered the same practical question. Correction learning was not the right starting point here, because neither baseline captured affinity well enough to make the residual easier to learn than absolute binding affinity itself. PRODIGY was weak on the harder subsets, and MM/GBSA was badly calibrated on the present benchmark. Later work therefore focused on predicting absolute binding affinity directly.

A.2 The Cα contact-map approach

An earlier model family represented complexes as fixed two-dimensional tensors built from Cα–Cα contact and distance channels, followed by a convolutional variational autoencoder and regression model. This image-like representation was appealing because related ideas have worked in protein–ligand learning [19]. The archived results, however, were too unstable for the present task. Some variants reduced RMSE relative to raw PRODIGY on small subsets, yet the same runs often had weak or negative test correlations and varied sharply across seeds. Table A.4 summarises representative test-set results from the direct-latent regression path built on that contact-map representation.

All runs in Table A.4 share the same basic contact-map representation and the same set-based edge encoder. The variants differ in checkpoint choice, random seed, supervision strategy, downstream regression, and the breadth of MD-frame coverage. That spread is useful because it shows the character of the branch: there were hints of lower error on

Variant	Test n	Direct-latent RMSE	Direct-latent Pearson r
Basic set-edge contact-map model	8	3.03	−0.065
24-epoch latent checkpoint	12	2.80	−0.500
24-epoch checkpoint with unregularised downstream regression	12	2.76	−0.527
24-epoch checkpoint, alternative seed A	12	6.62	−0.124
24-epoch checkpoint, alternative seed B	12	3.17	0.462
Semi-supervised latent training	12	5.48	0.072
Broader MD-frame variant	12	4.23	0.087

Table A.4: Representative Cα contact-map regression runs on the archived TCR subset. Some variants reduced RMSE relative to raw PRODIGY, but the results were unstable and often had weak or negative test correlation.

some small test slices, but the direction was not robust enough to justify taking it forward.

Compared with the final residue-graph formulation, the contact-map representation threw away residue identity detail, side-chain context, and explicit graph structure. It still depended on a fixed interface crop and a fixed tensor ordering. The residue-graph formulation retained more structural information and connected more naturally to the MD-derived features used in the main chapters, so the work later shifted away from the contact-map path.

A.3 PPB-Affinity interface-attention augmentation

Another line of work kept more of the trajectory itself, with baseline, frame-level, and trajectory-level models together with an ensemble stage. It stayed closer to the PPB-Affinity benchmark family by reusing a geometry-aware interface-attention model and then asking two broader questions. First, could saved MD frames act as useful supervised augmentation, with each frame inheriting the parent complex affinity label? Second, could ordered frame embeddings support a trajectory-level predictor that preserved temporal order instead of collapsing the trajectory to a few averages? The explored routes therefore included a single-structure baseline, frame-level models trained on augmented MD examples, and trajectory-level models that passed ordered frame embeddings through a bidirectional gated recurrent unit (BiGRU) before final regression [1, 38, 39]. An ensemble stage was intended to combine the baseline, frame-level, and trajectory-level routes. Recent protein–ligand work had started to learn from MD frame sequences and to retain temporal order instead of reducing trajectories to simple averages [12], which made the same question worth asking for protein–protein affinity prediction.

This branch was not run through to the same full comparison as the residue-graph

study in the main chapters because project focus shifted toward graphs with dynamic features. The change reflected scope and priorities, not a judgement that the augmentation idea lacked promise. The archived materials remain relevant future work, especially for asking whether MD-frame augmentation or spatiotemporal models can improve graph-based protein–protein affinity prediction.

Appendix B

Complete Model Matrix

This appendix gives the complete 52-model evaluation matrix, sorted by mean test RMSE. Here *Sup.* denotes supervision status, and *Gap* denotes the mean test-minus-train RMSE gap. A CSV version of the same matrix is included in the repository results directory.

Appendix B. Complete Model Matrix

Table B.1: Complete 52-model evaluation matrix, sorted by mean test RMSE. Metrics are means over 10 outer held-out fits.

View	Model	Mode	Sup.	Target	z	RMSE	r	R^2	Gap
Full	Baseline	S	sup	–	–	2.3379	0.5064	0.2146	0.8021
Full	Baseline	SD	sup	–	–	2.4073	0.4802	0.1685	0.8414
Full	GraphVAE	S	semi	static-only	32	2.4292	0.4649	0.1537	0.7451
Full	GraphVAE	SD	semi	static-only	8	2.4364	0.4749	0.1478	1.1303
Full	GraphVAE	S	semi	static-only	8	2.4407	0.4507	0.1451	0.6921
Full	GraphVAE	S	semi	static-only	16	2.4407	0.4499	0.1438	0.6580
Full	GraphVAE	SD	semi	static+dynamic	16	2.4419	0.4503	0.1437	0.8952
Full	GraphVAE	SD	semi	static-only	32	2.4423	0.4669	0.1443	1.2169
Full	GraphVAE	SD	semi	static-only	16	2.4516	0.4629	0.1354	1.1312
Full	GraphVAE	SD	semi	static+dynamic	32	2.4687	0.4411	0.1233	0.8527
Full	GraphVAE	SD	semi	dynamic only	32	2.4740	0.4508	0.1231	1.0153
Full	GraphVAE	SD	semi	dynamic only	8	2.5168	0.4202	0.0920	0.9580
Full	GraphVAE	SD	semi	static+dynamic	8	2.5277	0.4092	0.0818	0.7819
Full	GraphVAE	SD	semi	dynamic only	16	2.5543	0.3954	0.0629	1.0628
Interface	Baseline	S	sup	–	–	2.5782	0.3451	0.0470	0.8090
Interface	Baseline	SD	sup	–	–	2.5829	0.3347	0.0463	0.6954
Full	GraphVAE	S	unsup	static-only	32	2.5906	0.2271	0.0401	0.0617
Interface	GraphVAE	S	semi	static-only	8	2.6041	0.3402	0.0282	0.8018
Full	GraphVAE	S	unsup	static-only	16	2.6052	0.1910	0.0303	0.0194
Full	GraphVAE	S	unsup	static-only	8	2.6069	0.1984	0.0286	0.0016
Interface	GraphVAE	S	semi	static-only	32	2.6113	0.3364	0.0207	0.7354
Interface	GraphVAE	S	semi	static-only	16	2.6180	0.3316	0.0166	0.7561
Interface	GraphVAE	SD	semi	static+dynamic	16	2.6238	0.3252	0.0146	0.7597
Full	GraphVAE	SD	unsup	static-only	16	2.6313	0.1259	0.0102	0.0265
Full	GraphVAE	SD	unsup	static-only	8	2.6322	0.1249	0.0103	0.0124
Interface	GraphVAE	S	unsup	static-only	32	2.6324	0.1211	0.0102	0.0118
Full	GraphVAE	SD	unsup	static+dynamic	32	2.6328	0.1255	0.0093	0.0610
Full	GraphVAE	SD	unsup	dynamic only	32	2.6353	0.1100	0.0074	0.0364
Full	GraphVAE	SD	unsup	static-only	32	2.6365	0.1813	0.0039	0.1021
Interface	GraphVAE	SD	semi	dynamic only	16	2.6366	0.3298	0.0032	0.9460
Interface	GraphVAE	S	unsup	static-only	8	2.6427	0.0790	0.0025	0.0062
Full	GraphVAE	SD	unsup	dynamic only	16	2.6441	0.0981	0.0013	0.0292
Interface	GraphVAE	S	unsup	static-only	16	2.6454	0.0774	0.0003	0.0157
Full	GraphVAE	SD	unsup	static+dynamic	8	2.6454	0.0719	0.0006	0.0048
Interface	GraphVAE	SD	semi	static-only	16	2.6455	0.3362	-0.0015	1.1836

View	Model	Mode	Sup.	Target	z	RMSE	r	R^2	Gap
Full	GraphVAE	SD	unsup	dynamic only	8	2.6461	0.0706	0.0000	0.0013
Interface	GraphVAE	SD	unsup	static-only	8	2.6473	0.0618	-0.0010	0.0129
Interface	GraphVAE	SD	unsup	static+dynamic	8	2.6481	0.0552	-0.0016	0.0056
Interface	GraphVAE	SD	unsup	static-only	32	2.6530	0.0896	-0.0051	0.0451
Interface	GraphVAE	SD	unsup	static+dynamic	32	2.6534	0.0268	-0.0056	0.0222
Full	GraphVAE	SD	unsup	static+dynamic	16	2.6542	0.0582	-0.0060	0.0286
Interface	GraphVAE	SD	unsup	dynamic only	8	2.6552	0.0293	-0.0071	0.0097
Interface	GraphVAE	SD	unsup	dynamic only	16	2.6554	0.0235	-0.0070	0.0151
Interface	GraphVAE	SD	semi	dynamic only	8	2.6557	0.3279	-0.0116	1.0207
Interface	GraphVAE	SD	unsup	static+dynamic	16	2.6557	0.0209	-0.0073	0.0127
Interface	GraphVAE	SD	unsup	static-only	16	2.6575	0.0395	-0.0088	0.0353
Interface	GraphVAE	SD	unsup	dynamic only	32	2.6589	0.0347	-0.0099	0.0347
Interface	GraphVAE	SD	semi	static-only	8	2.6684	0.3270	-0.0217	1.1249
Interface	GraphVAE	SD	semi	static-only	32	2.6705	0.3185	-0.0230	1.2466
Interface	GraphVAE	SD	semi	static+dynamic	8	2.6743	0.3039	-0.0243	0.8622
Interface	GraphVAE	SD	semi	dynamic only	32	2.6855	0.3076	-0.0322	1.0470
Interface	GraphVAE	SD	semi	static+dynamic	32	2.6974	0.2941	-0.0411	0.9357

Appendix C

Repository Layout

The source code for the study is hosted at `https://github.com/RohanDevereux/Bridging`

The main repository layout is:

- Molecular-dynamics preparation and simulation:
  `https://github.com/RohanDevereux/Bridging/tree/main/src/bridging/MD`
- Residue-graph preparation:
  `https://github.com/RohanDevereux/Bridging/tree/main/src/bridging/graphvae/prep`
- GraphVAE, supervised baseline, and latent-ridge evaluation:
  `https://github.com/RohanDevereux/Bridging/tree/main/src/bridging/graphvae/ml`
- Command-line runners for the final evaluation:
  `https://github.com/RohanDevereux/Bridging/tree/main/src/bridging/graphvae/runners`
- Shared affinity-conversion and dataset utilities:
  `https://github.com/RohanDevereux/Bridging/tree/main/src/bridging/utils`
- Curated datasets used in the main analysis:
  `https://github.com/RohanDevereux/Bridging/tree/main/src/bridging/processedData`
- Final 52-configuration matrix summary:
  `https://github.com/RohanDevereux/Bridging/blob/main/RESULTS/final_resample_matrix_summary.csv`
- Saved subgroup analysis from the raw outer-fold prediction files:
  `https://github.com/RohanDevereux/Bridging/tree/main/RESULTS/final_resample_subgroups`

Appendix C. Repository Layout

- Subgroup-export script:
  `https://github.com/RohanDevereux/Bridging/blob/main/src/bridging/graphvae/tools/export_final_resample_subgroups.py`
- Archived branches discussed in Appendix A:
  `https://github.com/RohanDevereux/Bridging/tree/main/ARCHIVE`

Bibliography

[1] Huaqing Liu, Peiyi Chen, Xiaochen Zhai, Ku-Geng Huo, Shuxian Zhou, Lanqing Han, and Guoxin Fan. PPB-Affinity: Protein-protein binding affinity dataset for AI-based protein drug discovery. *Scientific Data*, 11(1):1316, 2024. doi: 10.1038/s41597-024-03997-4.

[2] Zhongliang Guo and Rui Yamaguchi. Machine learning methods for protein-protein binding affinity prediction in protein design. *Frontiers in Bioinformatics*, 2:1065703, 2022. doi: 10.3389/fbinf.2022.1065703.

[3] Ashley E. Modell, Sarah L. Blosser, and Paramjit S. Arora. Systematic targeting of protein-protein interactions. *Trends in Pharmacological Sciences*, 37(8):702–713, 2016. doi: 10.1016/j.tips.2016.05.008.

[4] Zoe Cournia, Bryce Allen, and Woody Sherman. Relative binding free energy calculations in drug discovery: Recent advances and practical considerations. *Journal of Chemical Information and Modeling*, 57(12):2911–2937, 2017. doi: 10.1021/acs.jcim.7b00564.

[5] Till Siebenmorgen and Martin Zacharias. Computational prediction of protein–protein binding affinities. *Wiley Interdisciplinary Reviews: Computational Molecular Science*, 10(3):e1448, 2020. doi: 10.1002/wcms.1448.

[6] Katherine Henzler-Wildman and Dorothee Kern. Dynamic personalities of proteins. *Nature*, 450:964–972, 2007. doi: 10.1038/nature06522.

[7] Panagiotis L. Kastritis, Iain H. Moal, Howook Hwang, Zhiping Weng, Paul A. Bates, Alexandre M. J. J. Bonvin, and Joël Janin. A structure-based benchmark for protein-protein binding affinity. *Protein Science*, 20(3):482–491, 2011. doi: 10.1002/pro.580.

[8] Iain H. Moal, Rudi Agius, and Paul A. Bates. Protein-protein binding affinity prediction on a diverse set of structures. *Bioinformatics*, 27(21):3002–3009, 2011. doi: 10.1093/bioinformatics/btr513.

[9] Sandra Romero-Molina, Yasser B. Ruiz-Blanco, Joel Mieres-Perez, Mirja Harms, Jan Münch, Michael Ehrmann, and Elsa Sanchez-Garcia. PPI-Affinity: A web tool for the

prediction and optimization of protein-peptide and protein-protein binding affinity. *Journal of Proteome Research*, 21(8):1829–1841, 2022. doi: 10.1021/acs.jproteome.2c00020.

[10] Jun Xie, Youli Zhang, Ziyang Wang, Xiaocheng Jin, Xiaoli Lu, Shengxiang Ge, and Xiaoping Min. PPI-Graphomer: Enhanced protein-protein affinity prediction using pretrained and graph transformer models. *BMC Bioinformatics*, 26(1):116, 2025. doi: 10.1186/s12859-025-06123-2.

[11] Yaosen Min, Ye Wei, Peizhuo Wang, Xiaoting Wang, Han Li, Nian Wu, Stefan Bauer, Shuxin Zheng, Yu Shi, Yingheng Wang, Ji Wu, Dan Zhao, and Jianyang Zeng. From static to dynamic structures: Improving binding affinity prediction with graph-based deep learning. *Advanced Science*, 11(40):2405404, 2024. doi: 10.1002/advs.202405404.

[12] Pierre-Yves Libouban, Camille Parisel, Maxime Song, Samia Aci-Sèche, Jose C. Gómez-Tamayo, Gary Tresadern, and Pascal Bonnet. Spatio-temporal learning from molecular dynamics simulations for protein–ligand binding affinity prediction. *Bioinformatics*, 41(8):btaf429, 2025. doi: 10.1093/bioinformatics/btaf429.

[13] Renxiao Wang, Xiaolu Fang, Yipin Lu, and Shaomeng Wang. The PDBbind database: Collection of binding affinities for protein–ligand complexes with known three-dimensional structures. *Journal of Medicinal Chemistry*, 47(12):2977–2980, 2004. doi: 10.1021/jm030580l.

[14] Yong Xiao Yang, Pan Wang, and Bao Ting Zhu. Binding affinity prediction for antibody-protein antigen complexes: A machine learning analysis based on interface and surface areas. *Journal of Molecular Graphics and Modelling*, 118:108364, 2023. doi: 10.1016/j.jmgm.2022.108364.

[15] David Graber, Peter Stockinger, Fabian Meyer, Siddhartha Mishra, Claus Horn, and Rebecca Buller. Resolving data bias improves generalization in binding affinity prediction. *Nature Machine Intelligence*, 7(10):1713–1725, 2025. doi: 10.1038/s42256-025-01124-5.

[16] Glenn M. Torrie and John P. Valleau. Nonphysical sampling distributions in Monte Carlo free-energy estimation: Umbrella sampling. *Journal of Computational Physics*, 23(2):187–199, 1977. doi: 10.1016/0021-9991(77)90121-8.

[17] Iva Massova and Peter A. Kollman. Combined molecular mechanical and continuum solvent approach (MM-PBSA/GBSA) to predict ligand binding. *Perspectives in Drug Discovery and Design*, 18(1):113–135, 2000. doi: 10.1023/A:1008763014207.

[18] Anna Vangone and Alexandre M. J. J. Bonvin. Contacts-based prediction of binding affinity in protein-protein complexes. *eLife*, 4:e07454, 2015. doi: 10.7554/eLife.07454.

[19] Marta M. Stepniewska-Dziubinska, Piotr Zielenkiewicz, and Pawel Siedlecki. Development and evaluation of a deep learning model for protein–ligand binding affinity prediction. *Bioinformatics*, 34(21):3666–3674, 2018. doi: 10.1093/bioinformatics/bty374.

[20] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. In *International Conference on Learning Representations*, 2014. URL `https://arxiv.org/abs/1312.6114`. Originally released as arXiv:1312.6114.

[21] Thomas N. Kipf and Max Welling. Variational graph auto-encoders. *arXiv preprint arXiv:1611.07308*, 2016. URL `https://arxiv.org/abs/1611.07308`.

[22] Martin Simonovsky and Nikos Komodakis. GraphVAE: Towards generation of small graphs using variational autoencoders. In *Artificial Neural Networks and Machine Learning – ICANN 2018*, volume 11139 of *Lecture Notes in Computer Science*, pages 412–422, Cham, 2018. Springer. doi: 10.1007/978-3-030-01418-6_41.

[23] Rafael Gómez-Bombarelli, Jennifer N. Wei, David Duvenaud, José Miguel Hernández-Lobato, Benjamín Sánchez-Lengeling, Dennis Sheberla, Jorge Aguilera-Iparraguirre, Timothy D. Hirzel, Ryan P. Adams, and Alán Aspuru-Guzik. Automatic chemical design using a data-driven continuous representation of molecules. *ACS Central Science*, 4(2):268–276, 2018. doi: 10.1021/acscentsci.7b00572.

[24] Hao Tian, Xi Jiang, Francesco Trozzi, Sian Xiao, Eric C. Larson, and Peng Tao. Explore protein conformational space with variational autoencoder. *Frontiers in Molecular Biosciences*, 8:781635, 2021. doi: 10.3389/fmolb.2021.781635.

[25] OpenMM Developers. PDBFixer, 2026. URL `https://github.com/openmm/pdbfixer`. Software repository; accessed 2026-03-25.

[26] Peter Eastman, Raimondas Galvelis, Raúl P. Peláez, Charlles R. A. Abreu, Stephen E. Farr, Emilio Gallicchio, Anton Gorenko, Michael M. Henry, Frank Hu, Jing Huang, Andreas Krämer, Julien Michel, Joshua A. Mitchell, Vijay S. Pande, João PGLM Rodrigues, Jaime Rodriguez-Guerra, Andrew C. Simmonett, Sukrit Singh, Jason Swails, Philip Turner, Yuanqing Wang, Ivy Zhang, John D. Chodera, Gianni De Fabritiis, and Thomas E. Markland. OpenMM 8: Molecular dynamics simulation with machine learning potentials. *The Journal of Physical Chemistry B*, 128(1):109–116, 2024. doi: 10.1021/acs.jpcb.3c06662.

[27] James A. Maier, Carmenza Martinez, Koushik Kasavajhala, Lauren Wickstrom, Karl E. Hauser, and Carlos Simmerling. ff14SB: Improving the accuracy of protein side chain and backbone parameters from ff99SB. *Journal of Chemical Theory and Computation*, 11(8):3696–3713, 2015. doi: 10.1021/acs.jctc.5b00255.

[28] Lee-Ping Wang, Todd J. Martinez, and Vijay S. Pande. Building force fields: An automatic, systematic, and reproducible approach. *The Journal of Physical Chemistry Letters*, 5(11):1885–1891, 2014. doi: 10.1021/jz500737m.

[29] Giulia Crocioni, Dani L. Bodor, Coos Baakman, Farzaneh M. Parizi, Daniel-T. Rademaker, Gayatri Ramakrishnan, Sven A. van der Burg, Dario F. Marzella, João M. C. Teixeira, and Li C. Xue. DeepRank2: Mining 3D protein structures with geometric deep learning. *Journal of Open Source Software*, 9(94):5983, 2024. doi: 10.21105/joss.05983.

[30] Robert T. McGibbon, Kyle A. Beauchamp, Matthew P. Harrigan, Christoph Klein, Jason M. Swails, Carlos X. Hernández, Christian R. Schwantes, Lee-Ping Wang, Thomas J. Lane, and Vijay S. Pande. MDTraj: A modern open library for the analysis of molecular dynamics trajectories. *Biophysical Journal*, 109(8):1528–1532, 2015. doi: 10.1016/j.bpj.2015.08.015.

[31] Matthias Fey and Jan E. Lenssen. Fast graph representation learning with PyTorch Geometric. arXiv preprint arXiv:1903.02428, 2019. URL `https://arxiv.org/abs/1903.02428`.

[32] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations*, 2019. URL `https://openreview.net/forum?id=ryGs6iA5Km`.

[33] Weihua Hu, Bowen Liu, Joseph Gomes, Marinka Zitnik, Percy Liang, Vijay Pande, and Jure Leskovec. Strategies for pre-training graph neural networks. In *International Conference on Learning Representations*, 2020. URL `https://openreview.net/forum?id=HJlWWJSFDH`.

[34] Samuel R. Bowman, Luke Vilnis, Oriol Vinyals, Andrew Dai, Rafal Jozefowicz, and Samy Bengio. Generating sentences from a continuous space. In *Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning*, pages 10–21, Berlin, Germany, 2016. Association for Computational Linguistics. doi: 10.18653/v1/K16-1002. URL `https://aclanthology.org/K16-1002/`.

[35] Casper Kaae Sønderby, Tapani Raiko, Lars Maaløe, Søren Kaae Sønderby, and Ole Winther. Ladder variational autoencoders. In *Advances in Neural Infor-*

mation Processing Systems 29, 2016. URL https://papers.nips.cc/paper/6275-ladder-variational-autoencoders.

[36] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. URL https://jmlr.org/papers/v12/pedregosa11a.html.

[37] Ron Kohavi. A study of cross-validation and bootstrap for accuracy estimation and model selection. In *Proceedings of the 14th International Joint Conference on Artificial Intelligence*, pages 1137–1143, 1995. URL https://www.ijcai.org/Proceedings/95-2/Papers/016.pdf.

[38] Mike Schuster and Kuldip K. Paliwal. Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45(11):2673–2681, 1997. doi: 10.1109/78.650093.

[39] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pages 1724–1734, 2014. doi: 10.3115/v1/D14-1179.